Improving Message Reliability in Wireless IoT Networks Using the MQTT Protocol: Theoretical Analysis and Experimental Validation

SADEEL QZIEH

Wireless Networks Course

***Abstract*—Wireless Internet of Things (IoT) devices often operate over links affected by interference, coverage gaps, mobility, and temporary outages. These conditions create a practical challenge: measurements may be missed, duplicated, or delayed before reaching a monitoring application. This paper explains what Message Queuing Telemetry Transport (MQTT) is, why it is used, where and when it is appropriate, and how it is deployed. We then formulate a concrete problem involving environmental sensors on a wireless smart farm and propose a theoretical reliability-oriented architecture. The design combines MQTT Quality of Service (QoS) 1, persistent sessions for eligible subscriber messages, application-side store-and-forward buffering for disconnected publishers, automatic reconnection, and application-level deduplication. We present architecture and sequence diagrams, configuration guidance, a worked outage example, trade-off analysis, and explicit limitations. A small controlled broker-outage demonstration was also implemented in Python with Eclipse Mosquitto, SQLite buffering, MQTT QoS 1, and subscriber acknowledgments. In this single demonstration, 20 of 20 generated readings were received after broker recovery (100% observed delivery). A basic unbuffered client failed to connect during the outage; no quantitative baseline delivery ratio was measured. The study demonstrates how MQTT mechanisms and careful application design can improve recoverability under intermittent wireless connectivity without implying that MQTT alone guarantees delivery during all failures.**

***Index Terms*—MQTT, wireless networks, Internet of Things, publish/subscribe, QoS, intermittent connectivity, reliability.**

I. Introduction

Wireless sensor networks and Internet of Things (IoT) systems exchange small but important measurements over Wi-Fi, cellular, and other wireless links. Unlike fixed wired connections, wireless links can experience fading, interference, roaming delays, power constraints, and temporary loss of connectivity. A monitoring application that misses a temperature alarm or receives a duplicated command can make incorrect decisions.

Message Queuing Telemetry Transport (MQTT) is an OASIS-standardized messaging protocol built around a publish/subscribe communication model [1], [2]. Its compact control packets, broker-mediated distribution, and configurable delivery semantics make it attractive for telemetry. MQTT is an application-layer protocol; in common deployments it runs over TCP/IP, while the wireless technology provides the underlying connectivity. MQTT does not repair radio signals or eliminate packet loss at the physical layer.

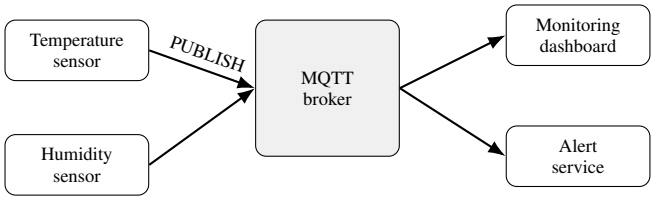


Fig. 1. MQTT publish/subscribe architecture: multiple publishers and subscribers communicate through a broker.

Research question: How can a wireless IoT monitoring system reduce avoidable loss of sensor measurements during short disconnections while managing duplicate messages and overhead?

Contributions: (i) a structured explanation of MQTT using the five questions requested for the project; (ii) a specific failure scenario and requirements; (iii) a proposed end-to-end recovery design; and (iv) a transparent theoretical comparison that distinguishes protocol guarantees from implementation assumptions.

II. What Is MQTT?

MQTT is a lightweight messaging protocol in which *clients* publish messages to named *topics*, and a *broker* routes those messages to clients that subscribe to matching topic filters [1]. A publisher does not need to know the identity or network address of each subscriber. A topic such as `farm/zone1/temperature` organizes messages by location and data type. The payload can be text, JSON, or binary; MQTT does not require a specific payload schema.

Figure explanation: The diagram shows the publish/subscribe pattern: wireless sensors publish measurements to the central broker, which routes them to the monitoring dashboard and alert service. The sensors do not need direct connections to the receiving applications.

III. Why Is MQTT Used?

MQTT is useful because its fixed header can be as small as two bytes, and the publish/subscribe model avoids building a direct connection between every sensor and consumer [1]. It supports many-to-many distribution, retained messages for the most recent state, a Last Will message to signal unexpected

TABLE I
MQTT QoS SEMANTICS AND DESIGN IMPLICATIONS

QoS	Delivery semantics	Trade-off
0	At most once	Lowest MQTT handshake overhead; message may be lost.
1	At least once	PUBACK confirms transfer; duplicates are possible.
2	Exactly once	More protocol exchanges and state; higher overhead.

client disconnection, and selectable QoS levels. These features simplify event-driven telemetry, although payload size, TCP/TLS overhead, broker capacity, and message frequency still matter.

IV. Where Is MQTT Used?

Representative applications include smart homes (device states and automation), agriculture (soil and weather sensors), industrial IoT (machine telemetry), smart buildings (energy monitoring), logistics (asset status), and environmental monitoring. The same broker can distribute a sensor reading to dashboards, alarms, and data collectors. Security and safety requirements depend on the application, not simply on the choice of MQTT [3], [4].

V. When Should MQTT Be Used?

MQTT is appropriate for small, periodic or event-driven messages; devices with constrained resources; multiple independent consumers; and connections that may be intermittent. It is less suitable as the primary mechanism for high-rate video streaming or large bulk transfers. MQTT is not automatically the best choice when direct request/response semantics, hard real-time guarantees, or broker-free operation are mandatory. Selection should consider network quality, latency budget, message criticality, power consumption, and operational complexity.

VI. How Should MQTT Be Used?

A basic deployment requires a broker, clients, topics, and agreed payload formats. The following sequence describes the intended behavior [1], [3].

1) Provision a broker and configure TLS, authentication, and topic access control lists (ACLs).
2) Give each device a unique client identifier; connect over TCP/TLS (commonly port 8883 for TLS deployments).
3) Subscribe the monitoring service to `farm/+/temperature` with the desired subscription QoS.
4) Publish readings to `farm/zone1/temperature` with QoS 1 and an application-level reading identifier.
5) On network loss, keep unsent readings in bounded persistent local storage; reconnect with backoff and replay the backlog.
6) Deduplicate readings at the receiver and monitor broker, client, and queue health.

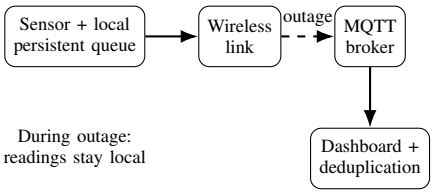


Fig. 2. Failure scenario: wireless outage separates the sensor from the broker; buffering must occur before broker receipt.

Illustrative payload: `(device=z1, id=105, temp=28.5)`. The identifier is an application design choice; MQTT does not automatically deduplicate readings at the application layer.

VII. Problem Statement and Scenario

Consider a smart farm with battery-powered wireless temperature sensors, a Wi-Fi gateway, a remote MQTT broker, and a monitoring dashboard. Each sensor generates one reading every 30 seconds. A temporary coverage gap prevents a sensor from reaching the broker for five minutes. With a simplistic QoS 0 publisher that discards measurements while disconnected, the dashboard may have missing data and delayed anomaly detection.

Problem: preserve measurements generated during a temporary wireless outage, deliver them when connectivity returns, and prevent repeated QoS 1 deliveries from appearing as distinct readings. A reliable solution must also bound storage usage and avoid excessive retransmission traffic.

Figure explanation: The wireless outage interrupts communication between the sensor and the MQTT broker. The proposed local queue retains measurements generated during this period so that the device can publish them when the connection is restored.

A. Why QoS Alone Is Insufficient

QoS 1 provides an MQTT-level acknowledgment mechanism for an established protocol exchange, but it does not automatically persist measurements that an application has not yet successfully submitted to the MQTT client. Broker-side persistent sessions mainly help retain eligible messages for disconnected *subscribers*; they do not replace local buffering at a disconnected publisher. QoS 1 also allows duplicates. These distinctions motivate the combined design [1].

VIII. Proposed Theoretical Solution

The design has five coordinated elements. **(1) Local store-and-forward:** write each measurement to a bounded, durable queue before attempting publication; remove it only after the application has confirmed its delivery policy has been satisfied. **(2) QoS 1:** publish with acknowledgment and retransmission behavior as defined by the MQTT protocol. **(3) Reconnection:** use exponential backoff with a maximum delay and jitter to avoid reconnect storms. **(4) Subscriber persistence:** use MQTT 5 session-expiry settings and appropriate subscription QoS so eligible broker-queued messages can

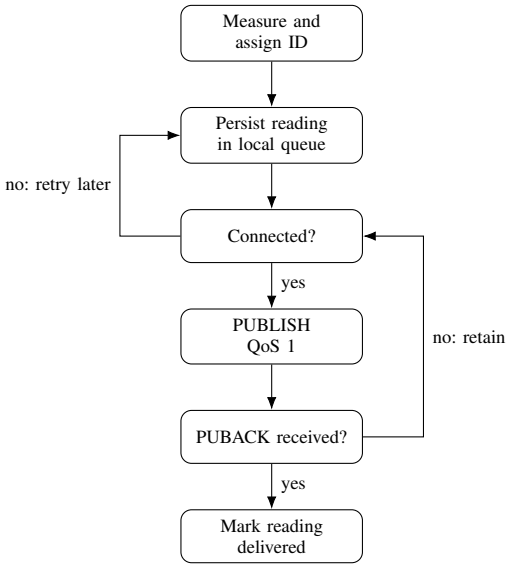


Fig. 3. Application-side store-and-forward recovery logic (conceptual).

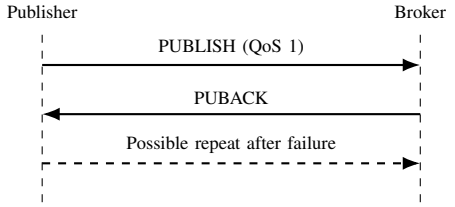


Fig. 4. QoS 1 acknowledgment; retransmission can lead to duplicate application data.

survive a short subscriber disconnection. **(5) Deduplication:** use a unique (device ID, reading ID) pair to ignore repeats at the application layer.

Figure explanation: This flowchart summarizes the proposed recovery procedure. Each reading is first stored locally; the client publishes it using QoS 1 when connected and retains it until the required acknowledgment is obtained. If the link fails, the client retries after reconnection.

A. MQTT QoS 1 Exchange

A successful QoS 1 transfer uses a PUBLISH packet and a PUBACK response [1]. If the sender does not obtain the acknowledgment, the transfer may be retried according to client/session behavior; the receiver can see duplicates. Therefore, deduplication must be based on an application reading ID, not merely on the MQTT packet identifier.

Figure explanation: The sequence diagram shows the PUBLISH and PUBACK exchange used by QoS 1. If the acknowledgment is lost, a repeated publication can occur; therefore, the receiving application must detect duplicate reading identifiers.

B. Recommended Configuration

Use TLS and credentials or client certificates; authorize topic access by device role. Set finite queue capacity and a retention policy. Choose message expiry and session expiry according to the maximum tolerable outage. A retained message can communicate the latest state to a new subscriber, but retained messages are not a historical event log. A Last Will can notify monitoring services that a device unexpectedly disconnected. Exact client behavior and durable storage depend on the implementation [1], [3].

C. Reliability Versus Resource Cost

QoS 1 adds acknowledgments and possible retransmissions. Persistent queues consume device flash or memory, while TLS and reconnects use energy and processing. An outage longer than the queue's capacity or message lifetime may still cause data loss. Thus the objective is *improved recoverability under stated assumptions*, not absolute end-to-end exactly-once delivery.

IX. Theoretical Analysis and Discussion

A. Worked Outage Example

Assume a sensor produces one measurement every $T = 30$ seconds, and the wireless link is unavailable for $D = 300$ seconds. The number of readings generated during the outage is approximately

$$N_{\text{outage}} = \frac{D}{T} = \frac{300}{30} = 10. \tag{1}$$

Equation (1) explanation: N_{outage} is the number of readings generated while the link is unavailable, D is the outage duration in seconds, and T is the sampling interval in seconds. With a 300-second outage and one reading every 30 seconds, ten readings are generated.

With no application buffering, these ten readings may be unavailable to the remote system. With a local queue holding at least ten readings, all ten can be retained *if* the device remains powered, local writes succeed, and retention limits are not exceeded. Once connectivity returns, the queue can replay the readings at QoS 1. The number delivered depends on broker reachability, acknowledgments, and retry policy. This example is illustrative arithmetic, not measured experimental performance.

If each stored reading occupies S bytes including metadata, a minimum raw queue budget is

$$B_{\min} = N_{\text{outage}} S, \tag{2}$$

Equation (2) explanation: $B_{\min}$ is the minimum raw buffer capacity in bytes, N_{outage} is the number of readings, and S is the assumed number of bytes stored per reading. The expression excludes storage overhead and safety margin.

excluding file-system overhead, wear-leveling, and safety margin. For $S = 128$ bytes, the ten readings require 1,280 bytes of raw storage. A production design must size for longer outages and account for flash endurance.

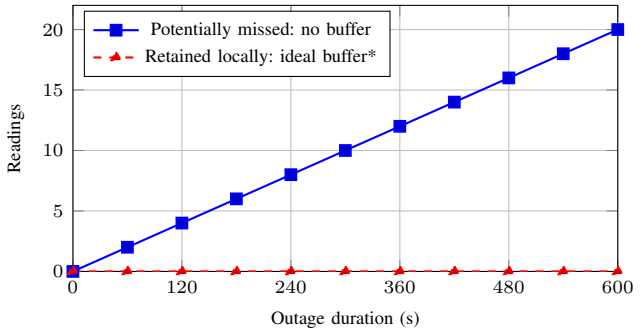


Fig. 5. Analytical illustration of readings not retained for later recovery. *The ideal-buffer line assumes sufficient durable storage, power, and successful writes; it does not imply guaranteed end-to-end delivery.

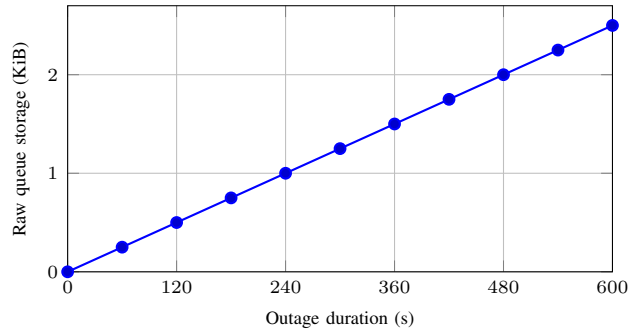


Fig. 6. Analytical raw storage requirement $B(D) = 128(D/30)$ bytes, converted to KiB (1024 bytes). File-system overhead and safety margin are excluded.

B. Analytical Graphs and Sensitivity Analysis

To make the assumptions explicit, let the sampling interval be T seconds and the outage duration be D seconds. For an outage aligned with the sampling grid, the number of generated readings is $N = D/T$. In the following plots, $T = 30$ s, the assumed local storage cost is $S = 128$ bytes per reading, and D ranges from 0 to 600 s. The figures are calculated from these assumptions, *not* measured from an MQTT deployment.

Figure explanation: The first curve grows with outage duration because more readings are produced while the unbuffered sensor is disconnected. The second curve represents zero locally unretained readings only for an ideal buffer with enough capacity and reliable writes; it is not a claim of zero end-to-end message loss.

Figure explanation: The graph illustrates a linear relationship between outage duration and raw buffer capacity. At 300 seconds, ten readings of 128 bytes require 1,280 bytes (1.25 KiB). Actual storage provisioning must also include metadata and a safety margin.

Figure 5 compares the *potential* loss of unbuffered readings against an idealized store-and-forward queue. The plotted zero for the buffered design refers only to local retention under stated assumptions, not successful remote delivery. Figure 6 shows the linear memory cost of supporting longer outages. These graphs highlight the design trade-off: recovery capability requires storage and later retransmission traffic. MQTT QoS does not alter the number of readings generated during an outage.

TABLE II
ANALYTICAL ASSUMPTIONS AND DERIVED VALUES

Parameter	Value or formula
Sampling interval T	30 s
Outage duration D	0–600 s
Storage per reading S	128 bytes (assumed)
Readings produced N	D/T (aligned grid)
Raw queue bytes B	$N \times S$
Example $D = 300$ s	$N = 10$, $B = 1280$ bytes

C. Interpretation of the Analytical Results

The two graphs answer different design questions. Figure 5 estimates how many measurements may be unavailable for recovery if a sensor simply discards readings while offline. Its increasing line is a count of *potentially missed readings*, not a measured packet-loss rate. The horizontal line represents an ideal local retention mechanism; it does not mean the broker or subscriber has already received those readings. Figure 6, in contrast, estimates the minimum raw memory required to preserve those readings. The graphs should therefore be interpreted together: retaining more measurements reduces the risk of unrecoverable application data, but requires a larger local queue.

For example, doubling the assumed outage from five to ten minutes increases the generated measurements from 10 to 20 and the raw queue requirement from 1.25 to 2.50 KiB. This linear relationship follows directly from the fixed sampling interval and fixed per-reading size. A real deployment should provision additional capacity for variable payloads, storage metadata, retries, and longer-than-expected outages. A device must also define what happens when the queue becomes full, such as dropping the oldest low-priority reading while preserving safety-critical alerts.

D. Why QoS 1 Alone Is Insufficient

MQTT QoS 1 is useful only once the client can participate in a protocol exchange with its broker. If a sensor is disconnected before publishing a new measurement, the broker cannot acknowledge data it never received. This is why the proposed solution places a durable application queue *before* the MQTT publication step. After reconnection, the application replays queued readings, and QoS 1 provides acknowledgment for each publication attempt. The receiver checks unique reading identifiers because a repeated publication may otherwise produce duplicate database entries. The combination addresses distinct failure points rather than relying on a single MQTT feature.

E. Why QoS 1 Instead of QoS 2?

QoS 0 is efficient but offers no MQTT acknowledgment, making it unsuitable when losing an important measurement is

TABLE III
SELECTION RATIONALE FOR MQTT QOS LEVELS

Level	Benefit	Limitation for this problem
0	Low protocol overhead	No acknowledgment; offline readings still need buffering.
1	Acknowledged transfer with moderate handshake	Duplicates require application filtering.
2	Exactly-once MQTT transfer semantics	Extra handshake/state; still needs offline buffering.

TABLE IV
QUALITATIVE COMPARISON (NOT BENCHMARK RESULTS)

Criterion	Basic QoS 0 design	Proposed design
Disconnected publisher	Measurements may be dropped	Local queue retains eligible readings
Delivery feedback	No MQTT acknowledgment	QoS 1 PUBACK
Duplicate readings	Not addressed	Application ID-based filtering
Disconnected subscriber	No session strategy	Session persistence for eligible queued data
Resource use	Lower	More storage, traffic, and energy
Failure recovery	Limited	Improved for temporary outages

unacceptable. QoS 1 uses PUBLISH and PUBACK to confirm transfer to the broker and is therefore a practical choice for periodic sensor telemetry. The trade-off is that repeated transmissions can produce duplicate application records. Our design addresses this by attaching a stable reading identifier and deduplicating at the receiver.

QoS 2 provides exactly-once delivery semantics for an MQTT protocol exchange using a four-step handshake (PUBLISH, PUBREC, PUBREL, PUBCOMP) [1]. It costs more control traffic and protocol state than QoS 1. Importantly, QoS 2 does not recover a measurement that was never created, persisted, or sent by an offline sensor, nor does it guarantee exactly-once database processing after application crashes. For this study, *QoS 1 plus durable buffering and deduplication* is selected to balance reliability with wireless bandwidth and device resources. QoS 2 may be justified for applications whose processing semantics and resources require it.

F. Comparison of Designs

G. Expected Benefits and Limitations

The combined design reduces a key gap: QoS 1 protects a protocol exchange, while a durable publisher queue protects readings before the network is available. Subscriber session persistence can further protect eligible downstream deliveries. Nonetheless, it does not prevent all losses: a dead sensor, full queue, expired message, broker storage failure, incorrect session settings, or permanent outage can defeat recovery. Duplicate suppression also requires durable receiver state and a suitable retention window.

H. Experimental Evaluation Plan and Remaining Work

Beyond the small broker-outage demonstration reported in Section XI, a larger repeatable experiment can use the open-source Eclipse Mosquitto broker [3], a simulated wireless sensor publisher, and a subscriber storing readings in a database. A network emulator or controlled Wi-Fi interruption would impose identical outages (30, 120, and 300 seconds) for a QoS 0 baseline and the proposed QoS 1 design. Both configurations should use the same sampling rate, payload size, broker hardware, and outage schedule. Repeat each scenario several times and report the mean and variability rather than selecting only a favorable run.

Application-level delivery ratio should be measured using *unique* generated and received reading IDs:

$$\text{Delivery Ratio} = \frac{N_{\text{unique,received}}}{N_{\text{generated}}} \times 100\%. \tag{3}$$

Equation explanation: $N_{\text{unique,received}}$ counts distinct generated readings received within the declared observation window; $N_{\text{generated}}$ counts all produced readings. Deduplication prevents QoS 1 retries from inflating the result.

The duplicate rate can be expressed as

$$\text{Duplicate Rate} = \frac{N_{\text{received}} - N_{\text{unique,received}}}{N_{\text{received}}} \times 100\%. \tag{4}$$

Equation explanation: N_{received} counts all received application messages, including repeats. This metric indicates how often the receiver must discard a duplicate; it is undefined when no messages are received.

Additional measures should include end-to-end latency (sensor timestamp to database commit), time to drain the queue after reconnection, peak buffer occupancy, and MQTT control-byte overhead. The observation window must extend beyond the outage to allow recovery traffic. These additional metrics were not measured in the small demonstration reported here. Published comparisons of IoT messaging protocols further motivate measuring latency and overhead instead of assuming that one protocol is universally superior [6], [7].

X. SECURITY AND OPERATIONAL CONSIDERATIONS

MQTT does not encrypt payloads by itself. Deployments should use TLS, secure credential management, topic ACLs, and protected broker storage. Reconnection backoff reduces repeated connection attempts during widespread outages. Monitoring should report queue saturation and age of oldest pending reading. Device clocks and message timestamps should be treated carefully: an old reading replayed after reconnection must not be mistaken for a fresh measurement.

XI. EXPERIMENTAL IMPLEMENTATION AND RESULTS

A. Implementation

A small local proof-of-concept was implemented on Windows using Python, the Paho MQTT client, and the Eclipse Mosquitto broker on localhost port 1883. A basic publisher/subscriber pair was contrasted qualitatively with a

TABLE V
OBSERVED BROKER-OUTAGE DEMONSTRATION (SINGLE TRIAL)

System		Observation
Basic client	MQTT	Broker connection failed during outage; no automatic backlog recovery was observed. Baseline delivery percentage not measured.
Reliable client	MQTT	Readings retained in SQLite while broker unavailable; after restart, 20/20 unique readings received and acknowledged; zero pending queue entries.

reliability-oriented pair. The reliable publisher writes 20 temperature readings with integer IDs to a SQLite queue before attempting broker communication. It publishes JSON readings at QoS 1 and removes a queued reading only after receiving a matching application-level acknowledgment from the reliable subscriber on a separate MQTT topic. The subscriber stores distinct reading IDs in a SQLite database and acknowledges processed readings. The demonstration uses a fixed five-second reconnect interval, rather than the exponential backoff recommended for production, and does not implement the persistent MQTT 5 subscriber sessions proposed in the theoretical architecture.

B. Outage Procedure

The Mosquitto service was stopped to simulate complete broker unavailability. While it was stopped, the basic publisher's connection attempt failed. In a separate run, the reliable publisher generated and locally retained its 20 readings while unable to connect. The broker was then restarted; the reliable publisher connected, published the queued readings, received subscriber acknowledgments, and emptied the local queue. The two runs establish observed behavior under a broker outage, not a statistically controlled wireless-link benchmark. No duration-controlled repeated trials or matched baseline message counts were recorded.

C. Observed Delivery

For the buffered run, the receiver database contained 20 unique IDs from 20 generated messages, giving an observed delivery ratio of $20/20 \times 100 = 100\%$ over the recovery observation window. No missing IDs were found in this run. These counts do not establish a general delivery probability or quantify a percentage improvement over the basic publisher. The subscriber has ID-based duplicate suppression, but the number of duplicate arrivals was not independently logged. Identifiers were reused across program executions, so the SQLite databases were reset before the independent trial.

D. Scope and Limitations

The broker service was stopped locally rather than imposing realistic radio fading or packet loss. Only one 20-message recovery trial was recorded; the demonstration did not measure latency, duplicate-arrival rate, network overhead, energy use, or queue saturation. The local MQTT connection was not secured with TLS. The implementation demonstrates application-level store-and-forward and acknowledgment, not

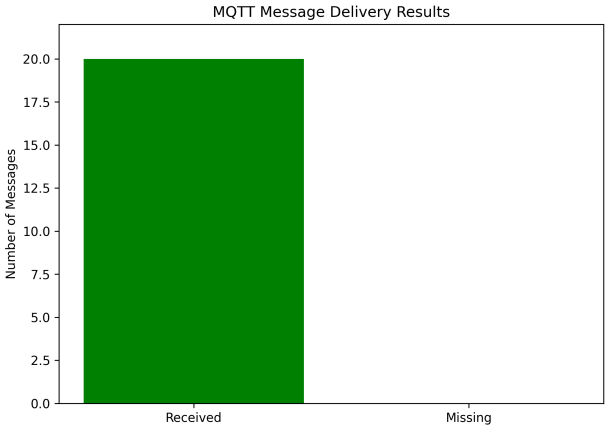


Fig. 7. Measured unique message count in the buffered outage-recovery demonstration: 20 received, zero missing (one trial).

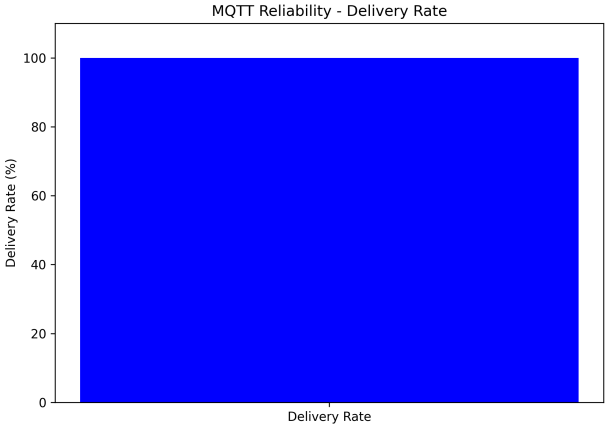


Fig. 8. Observed delivery ratio for the buffered run only (20/20); this is not a cross-system benchmark or a reliability guarantee.

all production features described in the theoretical proposal. Consequently, the analytical curves in the preceding section and the experimental observations here must be interpreted separately.

XII. CONCLUSION

This paper explained the MQTT protocol and its use in wireless IoT monitoring, then analyzed how local durable buffering, QoS 1, reconnection, and application-level acknowledgments can help recover measurements after a temporary broker outage. The theoretical queue-sizing calculations and sensitivity plots are analytical, not empirical. In a small Windows-based Mosquitto demonstration, the buffered publisher retained messages while the broker was stopped and subsequently obtained application acknowledgments for 20 of 20 readings after service restoration. The basic publisher could not connect during the outage, but a matched quantitative baseline delivery ratio was not established. The observed 100% recovery applies only to this one 20-message trial; it is not a general reliability guarantee. Future work should

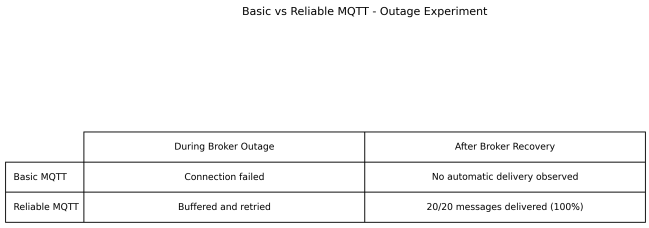


Fig. 9. Qualitative comparison of observed behavior during the broker-outage exercise; no numerical baseline delivery rate was established.

compare repeated, time-matched trials with realistic wireless impairment, different QoS levels, queue limits, latency, duplicate rates, and energy costs.

REFERENCES

[1] OASIS, "MQTT Version 5.0," 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[2] OASIS, "MQTT Version 3.1.1," 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[3] Eclipse Foundation, "Eclipse Mosquitto." [Online]. Available: https://mosquitto.org/

[4] HiveMQ, "MQTT Essentials." [Online]. Available: https://www.hivemq.com/mqtt/

[5] GitHub, "MQTT Open-Source Projects." [Online]. Available: https://github.com/topics/mqtt

[6] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, and C. K.-Y. Tan, "Performance evaluation of MQTT and CoAP via a common middleware," in *Proc. IEEE 9th International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, 2014.

[7] N. Naik, "Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP," in *Proc. IEEE International Systems Engineering Symposium (ISSE)*, 2017.